

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

TITLE:

MORSE CODE PLAYER WITH TEXT-TO-SPEECH AND AUDIO GENERATION USING PYTHON

AIM:

To design and implement a Python-based graphical user interface (GUI) application that converts text into Morse code and vice versa, produces real-time Morse audio beeps, saves Morse signals as an audio file, and provides text-to-speech functionality.

SOFTWARE USED:

- Python 3.x
- Tkinter (for GUI design)
- NumPy (for waveform generation)
- winsound (for tone generation)
- gTTS (Google Text-to-Speech)
- playsound (for playback)
- wave (for audio file handling)
- pyperclip (for clipboard functionality)

THEORY (BLOCK DIAGRAM / MODEL GRAPH):

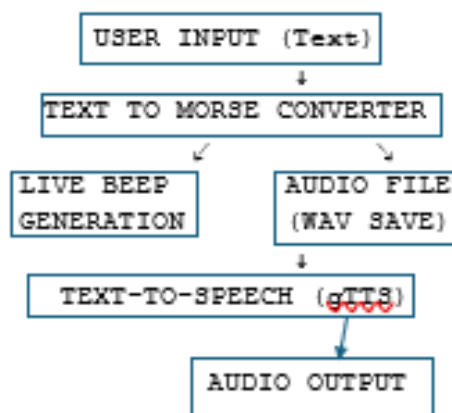
THEORY DESCRIPTION:

Morse code is an encoding technique that represents text characters using standardized sequences of dots (•) and dashes (–).

Each dot corresponds to a short tone, and each dash corresponds to a long tone. Spaces between letters and words are represented by pauses of specific durations.

In this project, Python's winsound library generates beeps to represent dots and dashes, while NumPy and wave are used to synthesize and save Morse tones as audio. The gTTS library provides speech synthesis to convert text input into human voice. The GUI interface built using Tkinter allows users to input, translate, listen, copy, or save Morse code efficiently.

BLOCK DIAGRAM:



PYTHON CODE:

```
1. import time
2. import winsound
3. import tkinter as tk
4. from tkinter import scrolledtext, filedialog, messagebox
5. import threading
6. import numpy as np
7. import wave
8. from gtts import gTTS
9. from playsound import playsound
10. import pyperclip
11. import os

12. # ----- Morse Code Dictionaries -----
13. TEXT_TO_MORSE = {
14. 'A': '-.-', 'B': '-...', 'C': '-.-.', 'D': '-..', 'E': '.',
15. 'F': '..-.', 'G': '--.', 'H': '....', 'I': '..', 'J': '-.-.-',
16. 'K': '-.-', 'L': '-.-.', 'M': '--', 'N': '-.', 'O': '---',
17. 'P': '-.-.', 'Q': '--.-', 'R': '-.-', 'S': '...', 'T': '-',
18. 'U': '..-', 'V': '...-', 'W': '-.-', 'X': '-.-.', 'Y': '-.-.-',
19. 'Z': '--.-', '0': '-----', '1': '.----', '2': '..---', '3': '...--',
20. '4': '....-', '5': '.....', '6': '-....', '7': '--...', '8': '-....',
21. '9': '-----', ' ': '/'
22. }
23. MORSE_TO_TEXT = {v: k for k, v in TEXT_TO_MORSE.items()}

24. # ----- Standard Morse Sound Settings -----
25. DOT_FREQ, DASH_FREQ = 800, 800
26. DOT_DUR, DASH_DUR = 100, 300 # in milliseconds

27. # ----- Morse Playback Function -----
28. def play_morse_live(message, output_widget):
29. message = message.strip()
30. is_morse = all(c in ".- /" for c in message)

31. output_widget.config(state='normal')

32. if is_morse:
```

```

33. # Morse → Text
34. words = message.split(' / ')
35. decoded = ""
36. for word in words:
37. for l in word.split():
    a. decoded += MORSE_TO_TEXT.get(l, '?')
38. decoded += ' '
39. output_widget.insert(tk.END, f"📖 Morse → Text: {decoded.strip()}\n", "bold")
40. else:
41. # Text → Morse
42. msg = message.upper()
43. morse = " ".join(TEXT_TO_MORSE.get(c, '?') for c in msg)
44. output_widget.insert(tk.END, f"🗨️ Input: {msg}\n", "bold")
45. output_widget.insert(tk.END, f"💠 Morse: {morse}\n\n", "italic")
46. output_widget.insert(tk.END, f"🎵 Playing live:\n\n")
47. output_widget.see(tk.END)
48. output_widget.update()

49. # ----- Optimized Delays -----
50. intra_symbol_delay = 0.05 # between dot/dash
51. inter_letter_delay = 0.1 # between letters
52. inter_word_delay = 0.15 # between words

53. for char in msg:
54. if char in TEXT_TO_MORSE:
    a. for symbol in TEXT_TO_MORSE[char]:
    b. start_index = output_widget.index(tk.END)
    c. output_widget.insert(tk.END, symbol + " ")
    d. output_widget.see(tk.END)
    e. output_widget.update()

    f. if symbol == '.':
        i. color = "#00BFFF"
        ii. winsound.Beep(DOT_FREQ, DOT_DUR)
    g. elif symbol == '-':
        i. color = "#1E90FF"
        ii. winsound.Beep(DASH_FREQ, DASH_DUR)
  
```

```
h. else:
    i. color = "gray"

    i. end_index = output_widget.index(tk.END)
    j. output_widget.tag_add("active", start_index, end_index)
    k. output_widget.tag_config("active", foreground=color, font=("Consolas", 16,
        "bold"))
    l. output_widget.update()
    m. time.sleep(intra_symbol_delay)
    n. output_widget.tag_remove("active", start_index, end_index)

    o. output_widget.insert(tk.END, " ")
    p. time.sleep(inter_letter_delay)
55. elif char == ' ':
    a. output_widget.insert(tk.END, "/" )
    b. output_widget.update()
    c. time.sleep(inter_word_delay)

56. output_widget.insert(tk.END, "\n\n✅ Done!\n", "done")
57. output_widget.tag_config("done", foreground="#00FF7F", font=("Arial", 13, "bold"))

58. output_widget.config(state='disabled')
59. output_widget.see(tk.END)

60. # ----- Morse Audio Generation -----
61. def save_morse_audio(message, filename="morse_output.wav"):
62. dot_dur_sec, dash_dur_sec = DOT_DUR / 1000, DASH_DUR / 1000
63. sample_rate = 44100
64. audio = np.array([], dtype=np.float32)

65. for char in message.upper():
66. if char in TEXT_TO_MORSE:
67. for symbol in TEXT_TO_MORSE[char]:
    a. if symbol == '.':
    b. t = np.linspace(0, dot_dur_sec, int(sample_rate * dot_dur_sec), False)
    c. tone = 0.5 * np.sin(DOT_FREQ * 2 * np.pi * t)
    d. elif symbol == '-':
    e. t = np.linspace(0, dash_dur_sec, int(sample_rate * dash_dur_sec), False)
```

```

f. tone = 0.5 * np.sin(DASH_FREQ * 2 * np.pi * t)
g. else:
h. tone = np.zeros(int(sample_rate * 0.1))
i. audio = np.concatenate((audio, tone, np.zeros(int(sample_rate * 0.05))))
68. audio = np.concatenate((audio, np.zeros(int(sample_rate * 0.2))))
69. elif char == ' ':
70. audio = np.concatenate((audio, np.zeros(int(sample_rate * 0.5))))

71. audio = np.int16(audio * 32767)
72. with wave.open(filename, 'w') as f:
73. f.setnchannels(1)
74. f.setsampwidth(2)
75. f.setframerate(sample_rate)
76. f.writeframes(audio.tobytes())

77. # ----- Text-to-Speech -----
78. def speak_text():
79. msg = entry.get().strip()
80. if not msg:
81. messagebox.showwarning("Warning", "Enter a message first!")
82. return
83. tts = gTTS(text=msg, lang='en')
84. tmp_file = "tts_temp.mp3"
85. tts.save(tmp_file)
86. threading.Thread(target=playsound, args=(tmp_file,), daemon=True).start()

87. # ----- Copy Output -----
88. def copy_output():
89. msg = entry.get().strip()
90. if not msg:
91. messagebox.showwarning("Warning", "Enter a message first!")
92. return
93. morse_text = " ".join(TEXT_TO_MORSE.get(c.upper(), '?') for c in msg)
94. pyperclip.copy(morse_text)
95. messagebox.showinfo("Copied", "Morse code copied to clipboard!")

96. # ----- GUI Handlers -----
97. def on_play(event=None):

```

```

98. msg = entry.get()
99. if msg.strip():
100.     threading.Thread(target=play_morse_live, args=(msg, output),
        daemon=True).start()

101.     def clear_all():
102.         entry.delete(0, tk.END)
103.         output.config(state='normal')
104.         output.delete("1.0", tk.END)
105.         output.config(state='disabled')

106.     def on_download():
107.         msg = entry.get().strip()
108.         if not msg:
109.             messagebox.showwarning("Warning", "Enter a message first!")
110.         return
111.         file_path = filedialog.asksaveasfilename(defaultextension=".wav",
        filetypes=[("WAV Audio", "*.wav")])
112.         if file_path:
113.             save_morse_audio(msg, filename=file_path)
114.             messagebox.showinfo("Saved", f"Morse audio saved to {file_path}")

115.     def decode_morse_audio():
116.         messagebox.showinfo("Info", "Audio decoding to text is not yet implemented.")

117.     # ----- GUI Setup -----
118.     root = tk.Tk()
119.     root.title("🎵 Morse Code Player + TTS + Copy")
120.     root.geometry("1050x780")
121.     root.config(bg="black")
122.     root.rowconfigure(2, weight=1)
123.     root.columnconfigure(0, weight=1)
124.     # Label
125.     tk.Label(root, text="💬 Enter message or Morse code (. - /):",
126.         font=("Arial", 18, "bold"), bg="black", fg="#1E90FF", pady=10).grid(row=0,
        column=0, sticky="ew")
127.     # Entry

```

```

128.     entry = tk.Entry(root, width=60, font=("Consolas", 20), justify="center",
        bg="#111", fg="#00BFFF",
        a. insertbackground="#00BFFF")
129.     entry.grid(row=1, column=0, sticky="ew", padx=10, pady=5)
130.     entry.focus_set()
131.     entry.bind("<Return>", on_play)
132.     # Output
133.     output = scrolledtext.ScrolledText(root, width=100, height=32, font=("Consolas",
        13), bg="#000", fg="#00FFFF",
        i. insertbackground="#00BFFF")
134.     output.grid(row=2, column=0, sticky="nsew", padx=10, pady=5)
135.     output.config(state='disabled')

136.     # Buttons
137.     btn_frame = tk.Frame(root, bg="black")
138.     btn_frame.grid(row=3, column=0, sticky="ew", pady=10)
139.     btn_frame.columnconfigure([0, 1, 2, 3, 4, 5], weight=1)

140.     tk.Button(btn_frame, text="▶ Play / Translate", command=on_play,
        font=("Arial", 16, "bold"), bg="#1E90FF",
        fg="white").grid(row=0, column=0, padx=5, sticky="ew")
141.     tk.Button(btn_frame, text="🗑 Clear", command=clear_all, font=("Arial", 16,
        "bold"), bg="#FF4500", fg="white").grid(
142.         row=0, column=1, padx=5, sticky="ew")
143.     tk.Button(btn_frame, text="🔊 Speak Text", command=speak_text, font=("Arial",
        16, "bold"), bg="#FF69B4",
        fg="white").grid(row=0, column=2, padx=5, sticky="ew")
144.     tk.Button(btn_frame, text="📁 Decode Audio → Text",
        command=decode_morse_audio, font=("Arial", 16, "bold"), bg="#FFD700",
        fg="black").grid(row=0, column=3, padx=5, sticky="ew")
145.     tk.Button(btn_frame, text="📁 Download Morse Audio",
        command=on_download, font=("Arial", 16, "bold"), bg="#32CD32",
        fg="white").grid(row=0, column=4, padx=5, sticky="ew")
146.     tk.Button(btn_frame, text="📄 Copy Output", command=copy_output,
        font=("Arial", 16, "bold"), bg="#30CD30",
        fg="white").grid(row=0, column=5, padx=5, sticky="ew")
147.     root.mainloop()
  
```

Project Working Principle and GUI:

The Morse Code Player project utilizes a GUI for easy, interactive encoding, decoding, and playback. The main application window, as shown below, allows users to enter text or Morse code, view live output, play Morse sounds, and use other features like copying and saving audio.

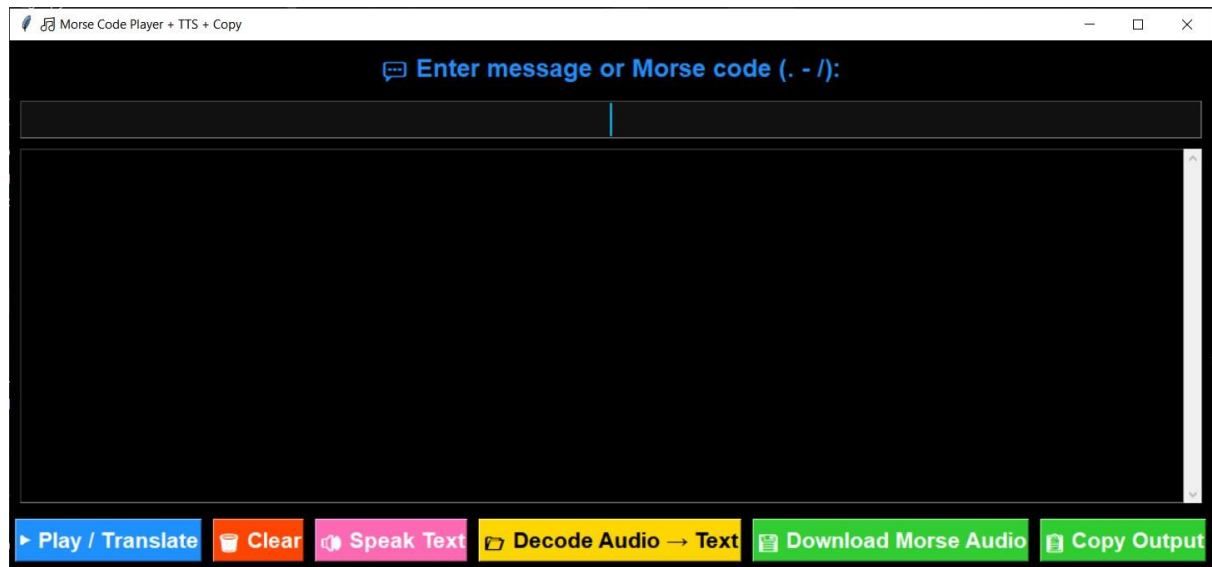


Figure 1: Application main user interface: ready for message input and operation selection.

EXPERIMENTAL OUTPUTS WITH SCREENSHOTS

Text to Morse and SOS Example

Below, the program is shown translating and playing the word HELLO and the

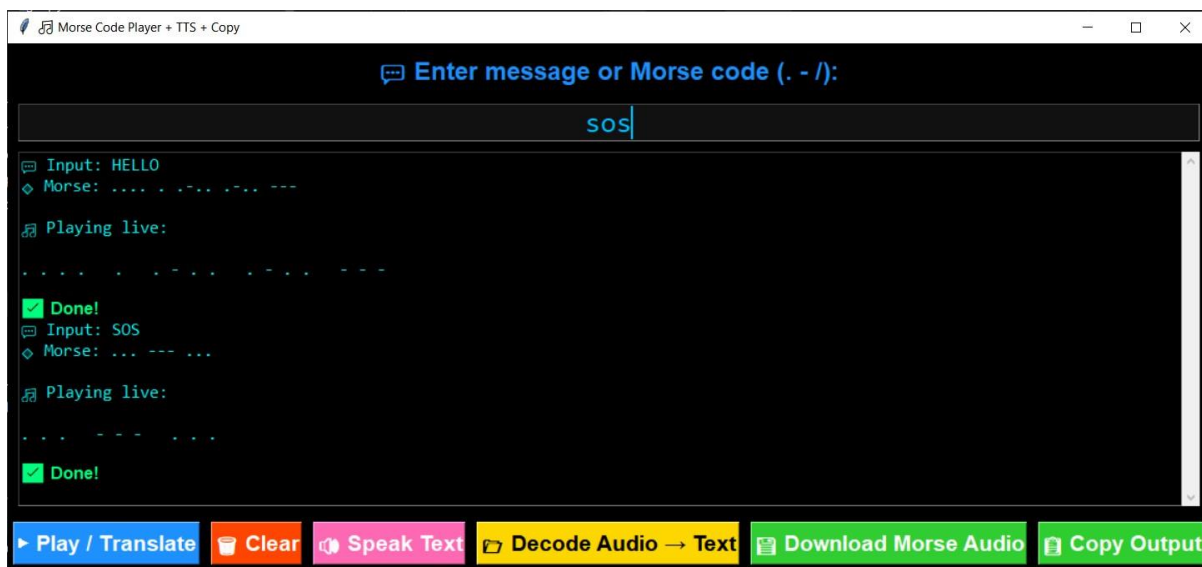


Figure 2: Text-to-Morse (HELLO) and SOS demonstration Playback and result log

emergency signal SOS. The output window shows Morse code translation, live playback feedback, and successful completion

Here is shown Morse code input (for HELLO) being successfully decoded back to human readable text:

Morse to Text Example:

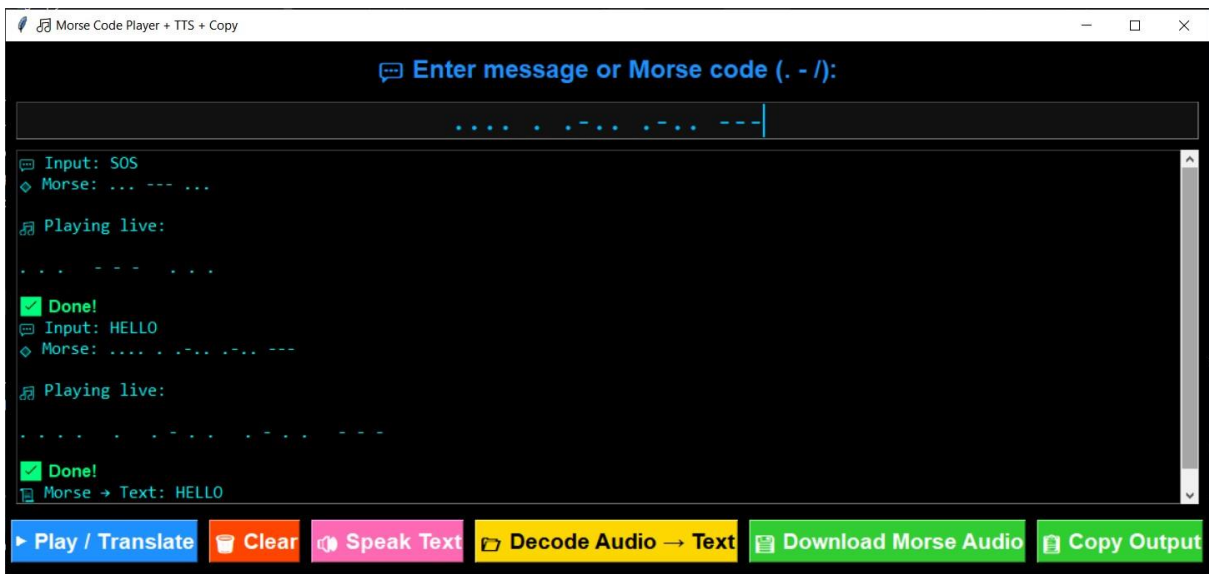


Figure 3: Morse-to-text decoding shown live for '..... . -.. .-.. ---' (HELLO).

Example 1:

Input: HELLO

Output Morse: -.. .-.. ---

- ✓ Real-time beeps generated for each dot and dash
- ✓ "HELLO" can be spoken aloud using Text-to-Speech
- ✓ Morse audio saved as morse_output.wav

Example 2:

Input Morse: -.. .-.. ---

Decoded Text: HELLO

Example 3 (Emergency Signal):

Input: SOS

Output Morse: ... --- ...

- ✓ Morse beeps for SOS pattern: dot-dot-dot (S), dash-dash-dash (O), dot-dot-dot (S)
- ✓ Audible short and long beeps clearly indicate the international distress signal



SUPPORTING DOCUMENT (MANUAL CALCULATION):

Character	Morse Code	Dot Duration (ms)	Dash Duration (ms)	Total Time (approx)
S	...	3×100	—	300 ms
O	---	—	3×300	900 ms
S	...	3×100	—	300 ms

Total Duration for “SOS” $\approx$ 1500 ms (1.5 seconds)

Character	Morse Code	Dot Duration (ms)	Dash Duration (ms)	Total Time (approx)
H		4×100	—	400 ms
E	.	100	—	100 ms
L	.-..	$2 \times 100 + 300$	—	500 ms
O	---	—	3×300	900 ms

Total Duration for “HELLO” $\approx$ 1900 ms (2 seconds)

RESULT:

The developed Python application successfully performs **bi-directional Morse code translation, real-time sound generation, audio saving, and text-to-speech playback.**

The Morse code for emergency signals such as **SOS (... --- ...)** is also correctly encoded and played audibly.

This project effectively demonstrates the integration of **signal processing, GUI design, and multimedia handling** using Python.

SUBMITTED BY,
SHAIK MUZAHID BASHA
230051601102
ECE - B